In [ ]:
```
#WITHOUT PIPLINE CHEKTHE MODEL
import pandas as pd
import joblib



# 1.Load
train_columns = joblib.load("x_train_columns.pkl")# this already x train  column lo
le_brand = joblib.load("brand_encoded.pkl")# alrady leble encoded data colum load
le_fuel = joblib.load("fuel_type_encoded.pkl")#alrady leble encoded data colum load
car_name_model  = joblib.load("target_mean.pkl")#alrady leble encoded data colum lo
model  = joblib.load("USED_CAR_price_model .pkl")# rf model for pridict load



#2 . inputs for pridict and if you give lower case same type all lower case /
#  no copy and past frome direct csv fill
new_car = {
    "brand": "maruti",
    "fuel_type": "petrol",
    "city": "kolkata",
    "kms_driven": 10344,
    "year_of_manufacture": 2019,
    "car_model_name":"suzuki alto 800 lxi"

}

#3. this do just transform like this not do fit_transform becouse aledy did this

if new_car["brand"] in le_brand.classes_:#2 le_brand is here
    brand_encoded = le_brand.transform([new_car["brand"]])[0]
else:
    # Not seen before, assign a default value (e.g., -1) or global mean
    brand_encoded = -1

if new_car["fuel_type"] in le_fuel.classes_:
    fuel_encoded = le_fuel.transform([new_car["fuel_type"]])[0]
else:
    fuel_encoded = -1

# this get metot use for target encoded and 1.car_name_model is here
target_model_name = car_name_model.get(new_car["car_model_name"], car_name_model.me





car_age = 2024 - new_car['year_of_manufacture']
#4.
# Create base dataframe "model_target_encoded":target_model_name/
#  when you save what have name same type have in input_df example  this  "model_ta
input_df = pd.DataFrame([{
    "kms_driven": new_car['kms_driven'],#3 is here enter
    "car_age": car_age,
```

```
    "brand_encoded": brand_encoded,
    "fuel_type_encoded": fuel_encoded,
    "model_target_encoded":target_model_name


}])




# Create all city columns = 0
for col in train_columns:  #one hot methot
    if col.startswith("city_"):
        input_df[col] = 0

# Set correct city = 1
city_col = "city_" + new_car['city']
if city_col in input_df.columns:
    input_df[city_col] = 1


#5

input_df = input_df.reindex(columns=train_columns, fill_value=0)#4 is here enter th

#6. final pritict
price = model.predict(input_df)[0] #5 is here enter
print(f"Predicted Price: ₹{price:,.0f}")
```

```
Predicted Price: ₹469,098
```

The script performs car price prediction using a trained machine learning model without using a pipeline. It starts by loading the required files such as the trained model, encoded label mappings for brand and fuel type, and the training column structure. These components ensure that the input data is transformed in the same way as during model training.

Next, the user provides input details for a new car, including brand, fuel type, city, kilometers driven, year of manufacture, and car model name. The categorical features such as brand and fuel type are converted into numerical format using pre-trained label encoders. If a value is not found in the encoder, a default value is assigned.

For the car model name, target encoding is applied using a precomputed mapping. If the model name is not available, the average value is used as a fallback. The car age is then calculated by subtracting the year of manufacture from the current year.

A new DataFrame is created with all the required features, including numerical and encoded values. City information is handled using one-hot encoding, where all city columns are initially set to zero, and the selected city is marked as one.

Finally, the input data is aligned with the training dataset structure to ensure consistency. The processed data is then passed into the trained model to predict the car price, and the

result is displayed in rupees.